05. API design

Status	Draft v1.0 for team review. Aligned to `architecture.md`: the **core** API carries the whole customer buying path including checkout and the Razorpay webhook; the **vendor-admin** API is the seller back-office and acts on orders and stock only by asking core
Date	2026-10-20
Depends on	`02-user-flows.md`, `architecture.md`, `04-database-design.md`
Feeds	`06-mvp-slice-and-migration.md`

Legend: `[As-is]` exists today on the Django backend (REST under `/v1`), `[Target]` the design, `[MVP]` built by Oct 30, `[Assumption]` needs a team decision.

Service tags: every endpoint is tagged **core** (Django, `api.shazlo.store`) or **vendor-admin** (FastAPI, `vendor-api.shazlo.store`). Internal routes are tagged **internal**.

1. Surfaces

Surface	Service	Style	Base	Clients	Status
App API	core	GraphQL	`api.shazlo.store/graphql`	Customer app (and the website's swipe demo)	`[Target]`; REST under `/v1` for the `[MVP]` (section 10)
Customer commerce routes (cart, checkout, orders, returns)	core	REST (MVP) / GraphQL (target)	`api.shazlo.store/v1/*`	Customer app	`[MVP]`
Public API (previews, config, serviceability, contact)	core	REST, no login	`api.shazlo.store/v1/public/*`	Website, app, deep-link previews	`[MVP]`
Payment webhook	core	REST, signed	`api.shazlo.store/v1/webhooks/razorpay`	Razorpay	`[MVP]` built, active only when the payments switch is on
Vendor application	vendor-admin	REST, no login	`vendor-api.shazlo.store/api/v1/public/vendor-applications`	Website form	`[MVP]`
Vendor portal API	vendor-admin	REST (FastAPI)	`vendor-api.shazlo.store/api/v1/vendor/*`	Vendor portal single-page app	`[MVP]`
Admin API	vendor-admin	REST (FastAPI)	`vendor-api.shazlo.store/api/v1/admin/*`	Admin pages; SQLAdmin at `admin.shazlo.store` covers data screens	`[MVP]`
Other webhooks (WhatsApp provider, courier)	vendor-admin	REST, signed	`vendor-api.shazlo.store/api/v1/webhooks/*`	Providers	`[Target]` (they only trigger commands to core)
Internal commands	core	REST, HMAC-signed	`/internal/v1/*` on core	Vendor-admin only	`[MVP]`; section 8
Internal events	both	REST, HMAC-signed	`/internal/v1/events` on each	The other service only	`[MVP]`; section 8
Health	both	REST	`/healthz`, `/readyz`	Monitoring	`[MVP]`

Why two APIs. The customer app and the vendor portal have different users, data, tokens and release rhythms. **Core owns money, stock and orders**, so everything that touches them is a core route; the vendor-admin API never changes them directly, it asks core through section 8. The app talks only to core; the portal and admin talk only to the vendor-admin service.

Why GraphQL for the app and REST for the rest. App screens need different slices of the same data (a product card, a product page, a cart line), which GraphQL serves in one request. The vendor portal, website forms, webhooks and the internal API are simple resource calls or fixed-format callbacks, where REST is the better fit (and FastAPI generates the OpenAPI file). **For the MVP the app keeps using REST** on core (section 10); the decision record is `adr/0002`, and it concerns **core only**.

2. Conventions

Topic	Rule
Format	REST: JSON, `snake_case`. GraphQL: `camelCase`
Ids	UUID strings
Money	Integer paise, field names end in `_paise` (REST) or `Paise` (GraphQL). Currency is always INR
Time	ISO 8601 in UTC
Pagination	Cursor based: `?cursor=<opaque>&limit=20` (maximum 50). Response has `items` and `next_cursor` (null at the end)
Auth: customer app (core)	`Authorization: Bearer <access token>`, issued by core, audience `core`. Access token 15 minutes, refresh token 30 days with rotation
Auth: vendor and admin (vendor-admin)	`Authorization: Bearer <access token>`, issued by the vendor-admin service, audience `vendor`, its own signing key. Access token 15 minutes, refresh token 7 days with rotation. A token from one service is **never** accepted by the other
Auth: internal	HMAC signature headers, section 8.1. No user tokens

Topic	Rule
Headers	`X-Request-Id` (returned on every response and **passed on to the other service** so one request can be traced across both), `X-App-Version` (sent by the app), `Idempotency-Key` (required on order placement, optional on other writes)
Errors (REST, both services)	`{"message": "...", "extra": {"code": "...", "fields": {...}}}` with the HTTP status below
Errors (GraphQL)	A GraphQL error with `extensions: {code, extra}`
Unknown or someone else's object	`404`, never `403`
Rate limits	429 with `Retry-After`; limits in section 9
Versioning	Additive changes in place; breaking changes get a new path version; GraphQL fields are deprecated first and removed only after old app versions are gone
Contracts	Core: committed `schema.graphql` (and the REST `openapi.json` for the MVP routes). Vendor-admin: `openapi.json` for the vendor, admin and public routes. **Internal API and event schemas** live in `contracts/` in the portal repo, with a pinned copy in the core repo (`architecture.md` section 10.2). CI fails when any of them is stale or has a breaking change

Error codes (`extra.code`; owner is the service that raises it):

Code	HTTP	Owner	When
`unauthenticated`	401	both	Missing, expired or wrong-audience token, or a bad internal signature
`not_found`	404	both	Unknown id, or not yours
`validation_error`	422	both	Bad input; `extra.fields` lists problems
`invalid_transition`	409	core (orders), vendor-admin (products, vendors)	Status change not allowed from the current status
`not_cancellable`	409	core	The order can no longer be cancelled by the customer
`stock_unavailable`	409	core	Not enough available stock; `extra.lines` lists the variants
`stock_below_reserved`	409	core	A vendor tried to set `on_hand` below what is reserved
`version_conflict`	409	both	`expected_version` is stale (inventory set, copies)
`not_serviceable`	422	core	Pincode outside the launch area
`vendor_not_approved`	403	core, vendor-admin	Vendor is not `approved` (a customer cannot buy from a suspended vendor)
`product_not_editable`	409	vendor-admin	Product is in review
`online_payments_disabled`	409	core	A checkout asked for `online` while the switch is off
`payment_pending`	409	core	A vendor tried to confirm an online order that is not yet paid
`file_too_large`	413	vendor-admin	Upload over the limit
`unsupported_media_type`	415	vendor-admin	Wrong file type
`rate_limited`	429	both	Too many requests
`idempotency_conflict`	409	both	Same key used with a different request body
`service_unavailable`	503	vendor-admin	The vendor-admin service could not reach core within 3 seconds for a command; safe to retry

The scaffold currently turns every expected error into HTTP 400; the API layer should map `extra.code` to the statuses above `[Target change]`.

3. App API (GraphQL, core) `[Target]`

Rules: production accepts **persisted queries** only; query depth limit 10 and alias limit 15 (already in `shazlo/api/schema.py`); the IDE and introspection are off outside development; every mutation that changes money or stock accepts an idempotency key.

Cart, stock, prices, orders and payments are **core's own data**: products and variants come from the catalog copy (`catalog_items`), availability from `inventory`, and `orderPlace` is a single core transaction. Core does not call the vendor-admin service to take an order.

3.1 Schema sketch

```
scalar UUID
scalar DateTime

enum Gender { WOMEN MEN OTHER UNSPECIFIED }
enum SwipeAction { LIKE DISLIKE SAVE CART ORDER }
enum OrderStatus { PLACED CONFIRMED SHIPPED DELIVERED CANCELLED RETURN_REQUESTED RETURNED REFUNDED }
enum PaymentMethod { COD UPI_ON_DELIVERY ONLINE }
enum PaymentStatus { PENDING PAID FAILED REFUNDED }

type Me { id: UUID!  isGuest: Boolean!  displayName: String  email: String  phone: String
          gender: Gender!  rewardsBalance: Int! }
type AuthPayload { accessToken: String!  refreshToken: String!  expiresIn: Int!  me: Me! }

type PublicConfig { minAppVersion: String!  onlinePaymentsEnabled: Boolean!
                    paymentMethods: [PaymentMethod!]!  launchCities: [String!]! }

type Vendor { id: UUID!  name: String!  slug: String!  logoUrl: String }
type ProductImage { url: String!  angle: String!  position: Int! }
type Variant { id: UUID!  size: String  color: String  pricePaise: Int!  mrpPaise: Int!  inStock: Boolean! }
type Product { id: UUID!  title: String!  description: String  vendor: Vendor  fabric: String
               gender: String!  images: [ProductImage!]!  variants: [Variant!]!
               minPricePaise: Int!  returnPolicy: String  buyable: Boolean!  externalUrl: String }

type FeedBatch { slateId: UUID!  products: [Product!]!  category: String  explanation: String }

type Closet { id: UUID!  name: String!  members: [ClosetMember!]!  items: [Product!]!  itemCount: Int! }
type ClosetMember { user: Me!  role: String! }

type CartLine { id: UUID!  variant: Variant!  product: Product!  quantity: Int! }
type Cart { lines: [CartLine!]!  subtotalPaise: Int! }

type Address { id: UUID!  recipientName: String!  phone: String!  line1: String!  line2: String
               landmark: String  city: String!  state: String!  pincode: String!  isDefault: Boolean! }

type OrderItem { title: String!  size: String  color: String  imageUrl: String
                 unitPricePaise: Int!  quantity: Int! }
type StatusStep { status: OrderStatus!  at: DateTime! }
type Order { id: UUID!  number: String!  status: OrderStatus!  paymentMethod: PaymentMethod!
             paymentStatus: PaymentStatus!  vendor: Vendor!  items: [OrderItem!]!
             subtotalPaise: Int!  deliveryFeePaise: Int!  totalPaise: Int!
             placedAt: DateTime!  paymentDueAt: DateTime  eta: DateTime  history: [StatusStep!]! }
type PaymentIntent { provider: String!  razorpayOrderId: String!  amountPaise: Int!  keyId: String! }
type OrderPlacePayload { orders: [Order!]!  payment: PaymentIntent }   # payment only for ONLINE

input FeedInput { gender: Gender  minPricePaise: Int  maxPricePaise: Int  fabric: [String!]
                  categories: [String!]  vendorIds: [UUID!]  source: String  closetId: UUID }
input SwipeInput { productId: UUID!  action: SwipeAction!  dwellMs: Int  slateId: UUID
                   position: Int  clientEventId: UUID! }
input SignUpInput { displayName: String!  phone: String  email: String  password: String!
                    gender: Gender  dateOfBirth: String }
input AddressInput { id: UUID  recipientName: String!  phone: String!  line1: String!
                     line2: String  landmark: String  city: String!  state: String!
                     pincode: String!  isDefault: Boolean }
input OrderPlaceInput { addressId: UUID!  paymentMethod: PaymentMethod!  note: String }

type Query {
  me: Me!
  publicConfig: PublicConfig!
  feed(input: FeedInput!): FeedBatch!
  product(id: UUID!): Product
  vendors(cursor: String, limit: Int): [Vendor!]!
  closets: [Closet!]!
  closet(id: UUID!): Closet
  cart: Cart!
  addresses: [Address!]!
  serviceability(pincode: String!): Boolean!
  orders(cursor: String, limit: Int): [Order!]!
  order(id: UUID!): Order
  serverVersion: String!
}

type Mutation {
  guestStart(deviceId: String): AuthPayload!
  signUp(input: SignUpInput!): AuthPayload!          # upgrades the signed-in guest
  login(identifier: String!, password: String!): AuthPayload!
  googleLogin(idToken: String!): AuthPayload!
  refreshToken(refreshToken: String!): AuthPayload!
  logout(refreshToken: String!): Boolean!
  accountDelete: Boolean!

  swipeRecord(input: SwipeInput!): Boolean!
  swipeRecordBatch(inputs: [SwipeInput!]!): Boolean!
```

```
  closetCreate(name: String!): Closet!
  closetAddItem(closetIds: [UUID!]!, productId: UUID!): [Closet!]!
  closetRemoveItem(closetId: UUID!, productId: UUID!): Closet!
  closetInvite(closetId: UUID!): String!            # returns a share token
  closetJoin(token: String!): Closet!
  closetLeave(closetId: UUID!): Boolean!
  closetAddToCart(closetId: UUID!, productIds: [UUID!]!): Cart!

  cartAdd(variantId: UUID!, quantity: Int!): Cart!
  cartSetQuantity(lineId: UUID!, quantity: Int!): Cart!
  cartRemove(lineId: UUID!): Cart!

  addressSave(input: AddressInput!): Address!
  addressDelete(id: UUID!): Boolean!

  orderPlace(input: OrderPlaceInput!, idempotencyKey: String!): OrderPlacePayload!   # one core transaction
  orderCancel(id: UUID!, reason: String): Order!
  returnRequest(orderId: UUID!, itemIds: [UUID!]!, reason: String!): Order!

  deviceRegister(pushToken: String!, platform: String!, appVersion: String): Boolean!
}
```

`orderPlace` returns a list because a cart with products from two vendors becomes two orders (one per vendor, linked by `checkout_group_id`); the optional `payment` is present only for `ONLINE`.

3.2 Operations behind each screen

Screen or action	Operations
First launch	`guestStart`, then `serverVersion` and `publicConfig`
Feed	`feed`, `swipeRecord` or `swipeRecordBatch`
Product details and share	`product`
Filters and brand explore	`feed` with `FeedInput`, `vendors`
Closets tab and closet page	`closets`, `closet`, `closetCreate`, `closetAddItem`, `closetRemoveItem`, `closetInvite`, `closetJoin`, `closetLeave`, `closetAddToCart`
Cart	`cart`, `cartAdd`, `cartSetQuantity`, `cartRemove`
Checkout	`addresses`, `addressSave`, `addressDelete`, `serviceability`, `publicConfig` (which payment methods to offer), `orderPlace`
Orders	`orders`, `order`, `orderCancel`, `returnRequest`
Account	`signUp`, `login`, `googleLogin`, `refreshToken`, `logout`, `accountDelete`, `me`, `deviceRegister`

3.3 Behaviour that must hold

- `feed` returns **10 products**: 7 from the user's strongest category, 3 from others; unseen, approved, with available stock (the `feed_eligible` flag), embedding present (`02`, flow C2). `explanation` is a short human-readable reason.
- `swipeRecord` is idempotent on `clientEventId`; it never returns user vectors.
- `signUp` works only for a signed-in guest.
- `closetJoin` takes a share token (a revocable invite), not a closet id.
- `orderPlace` **is one core transaction:** it checks the signed-in user, address, pincode and cart; reads prices and the vendor's commission rate from the catalog copy and stores the catalog versions used; locks the variant `inventory` rows in ascending id order and reserves stock; creates one order per vendor with items, reservations, status events and outbox rows; for `ONLINE` it also creates the Razorpay order and returns the `payment` payload; then clears the cart and records an `order` swipe. It returns `stock_unavailable` for any variant that is short, and `online_payments_disabled` when the switch is off. **It never calls the vendor-admin service**, so checkout works while that service is down.
- `orderCancel` is handled in core: allowed from `placed` or `confirmed`, it releases the reservation and, if the payment was captured, creates a refund; after `shipped` it returns `not_cancellable`.
- `orders` and `order` read core's own order tables.

4. Public API (REST, no login) `[MVP]`

Method and path	Service	Purpose	Key input	Success
`POST /api/v1/public/vendor-applications`	vendor-admin	Website vendor application (replaces the Google Form and the legacy Node service)	`brand_name`, `owner_name`, `phone`, `email`, `city`, `instagram_or_website`, `product_types[]`, `price_min_paise`, `price_max_paise`, `about`, a spam-check token	`201 {id, status: "applied"}`
`GET /v1/public/closet-invites/{token}` (MVP: `GET /v1/public/closets/{id}`)	core	Preview for the closet landing page	none	`{closet_name, item_coun previews[] (3 or 4 images)}`
`GET /v1/public/products/{id}`	core	Preview for the product landing page; approved items only; reads the catalog copy	none	`{title, vendor_name, price_paise, image_url}`

Method and path	Service	Purpose	Key input	Success
`GET /v1/public/config`	core	Minimum app version, launch cities, `online_payments_enabled`, `payment_methods`, feature flags	none	`{min_app_version, serviceable_pincodes_hir online_payments_enabled, payment_methods[], flags`
`GET /v1/serviceability?pincode=`	core	Is delivery available (launch pincode list)	`pincode`	`{serviceable}`
`POST /v1/public/contact`	core	Website contact form	`name`, `email`, `message`	`202`
`GET /healthz`	both	Liveness	none	`{status: "ok"}`
`GET /readyz`	both	Readiness	none	Core: `{status, postgres, redis_state}`. Vendor-admi `{status, postgres}`. `503` when degraded

`online_payments_enabled` is a **core setting** (the single switch); the vendor-admin service never decides it. `/readyz` does not fail because the other service is down.

5. Vendor portal API (REST, vendor-admin) `[MVP]`

All routes need a **vendor token** (audience `vendor`) and are scoped to the caller's vendor. Roles are those of `vendor_users`: `vendor_owner` can do everything; `vendor_staff` cannot manage the team, policy or statements. The React portal calls this API; its client is generated from the vendor-admin OpenAPI file. The vendor-admin service **reads** orders from its own copy (`vendor_orders`) and **changes** orders or stock only by calling core (section 8); each such route returns the new state from core's response.

5.1 Authentication

Method and path	Purpose	Input	Success
`POST /api/v1/vendor/auth/login`	Sign in	`identifier` (email or phone), `password`	`{access_token, refresh_token, expires_in, vendor, role}`
`POST /api/v1/vendor/auth/refresh`	Rotate tokens	`refresh_token`	New pair
`POST /api/v1/vendor/auth/logout`	Revoke the refresh family	`refresh_token`	`204`
`POST /api/v1/vendor/auth/accept-invite`	First password after approval or staff invite	`token`, `password`	Tokens
`POST /api/v1/vendor/auth/password/forgot`	Start a reset	`identifier`	`202` (always, to avoid revealing accounts)
`POST /api/v1/vendor/auth/password/reset`	Finish a reset	`token`, `password`	`204`

5.2 Profile, policy, team

Method and path	Role	Purpose
`GET /api/v1/vendor/me`	any	Vendor profile, status, caller's role
`PATCH /api/v1/vendor/profile`	owner	Edit name, about, contact, logo
`GET`, `PUT /api/v1/vendor/policy`	owner	Return policy (`returnable`, `return_window_days`, `return_shipping_paid_by`, `processing_hours`)
`GET`, `POST /api/v1/vendor/locations`; `PATCH`, `DELETE .../{id}`	owner	Pickup addresses
`GET`, `POST /api/v1/vendor/team`; `DELETE .../{id}`	owner	Invite or remove staff (`vendor_users`)

5.3 Products and stock

Method and path	Purpose	Key input	Success and rules
`GET /api/v1/vendor/products?status=&q=&cursor=`	List own products	filters	`{items[], next_cursor}`; stock shown from the last known core value
`POST /api/v1/vendor/products`	Create a **draft**	`title`, `description`, `category`, `gender`, `fabric`, `returnable`, `weight_grams`	`201` product
`GET`, `PATCH /api/v1/vendor/products/{id}`	Read or edit	fields	`409 product_not_editable` while in `pending_review`. **Decided:** editing title, description or photos of an `approved` product sends it back to review and takes it off the feed; price edits apply at once and publish an update
`POST /api/v1/vendor/products/{id}/submit`	Draft or rejected to `pending_review`	none	Needs 3 or more images and one active variant, else `validation_error`
`POST /api/v1/vendor/products/{id}/archive`, `.../restore`	Hide or bring back	none	Status changes recorded; archiving publishes `product.unpublished`

Method and path	Purpose	Key input	Success and rules
`POST /api/v1/vendor/products/{id}/variants`; `PATCH`, `DELETE .../variants/{variant_id}`	Sizes, price	`size`, `color`, `price_paise`, `mrp_paise`, `sku`	Price changes apply at once and publish `product.updated`
`PUT /api/v1/vendor/variants/{variant_id}/stock`; `PATCH /api/v1/vendor/variants/stock` (bulk)	**Set stock**	`on_hand`, `expected_version` (bulk: `[{variant_id, on_hand, expected_version}]`)	Calls core `PUT /internal/v1/inventory/...`; `409 stock_below_reserved` or `version_conflict` come from core; returns `{on_hand, reserved, available, version}`
`POST /api/v1/vendor/products/{id}/images/presign`	Get an upload URL (Cloudinary signed upload in the MVP)	`content_type`, `size_bytes`	`{upload_url, key, headers}`; limits on type and size
`POST /api/v1/vendor/products/{id}/images`	Register an uploaded image	`key`, `angle`, `position`	`201` image
`PATCH /api/v1/vendor/products/{id}/images/order`; `DELETE .../images/{image_id}`	Reorder or remove	positions	`200`

5.4 Bulk import

Method and path	Purpose	Success
`GET /api/v1/vendor/imports/template.csv`	Download the CSV template	File
`POST /api/v1/vendor/imports`	Upload a CSV (file or storage key)	`202 {job_id}`
`GET /api/v1/vendor/imports/{job_id}`	Job status	`{status, total_rows, ok_rows, error_rows, errors: [{row, field, message}]}`

Valid rows become `draft` products; stock columns in the CSV are applied through core once the product is approved. The vendor fixes the rest and re-uploads only those.

5.5 Orders, returns, statements

Method and path	Purpose	Rules
`GET /api/v1/vendor/orders?status=&cursor=`	Own orders (read copy)	Newest first
`GET /api/v1/vendor/orders/{id}`	Detail with items, address, payment method and status	Customer phone is shown only after `confirmed` `[Assumption]`
`POST /api/v1/vendor/orders/{id}/confirm`	`placed` to `confirmed`	**Calls core.** `409 payment_pending` for an unpaid `online` order, `invalid_transition`; returns the new order state
`POST /api/v1/vendor/orders/{id}/cancel`	Cancel with `reason`	**Calls core.** From `placed` or `confirmed` only; core releases stock
`POST /api/v1/vendor/orders/{id}/ship`	`confirmed` to `shipped`; optional `courier_name`, `tracking_ref`	**Calls core.** Core commits the stock reservation
`GET /api/v1/vendor/orders/{id}/events`	Status history	From the copy
`GET /api/v1/vendor/returns`; `POST .../returns/{id}/approve`, `.../decline`	Return decisions	**Calls core.** `decline` needs a reason
`GET /api/v1/vendor/payouts`; `GET .../payouts/{id}`	Monthly statements and lines	Built from the copy; commission shown per order (15% flat for pilot vendors)
`GET /api/v1/vendor/analytics/summary?from=&to=`	Views, likes, saves, cart adds, orders per product	Aggregates only `[Target]` (swipe data lives in core, so this needs an export from core)

`delivered` and "payment collected" are set by an admin (calls core) in the MVP and by the courier webhook in the target. A vendor who does not confirm within 4 business hours is reminded (core emits `order.confirmation_overdue`, vendor-admin emails), then core auto-cancels the order (decided).

6. Admin API (REST, vendor-admin) `[MVP]`

Role `admin` only, using an **admin token** (audience `vendor`, separate accounts from vendor users). Every action writes an `audit_log` row. Admin accounts are created by an engineer. SQLAdmin offers screens for vendors, products, users and audit; **order operations are portal admin pages that call this API, which calls core**.

Method and path	Purpose	Input
`POST /api/v1/admin/auth/login`, `.../refresh`, `.../logout`	Admin session	`email`, `password` (second factor `[Target]`)
`GET /api/v1/admin/vendors?status=`	Review queue	filters
`POST /api/v1/admin/vendors/{id}/approve`	`under_review` to `approved`; creates the owner `vendor_users` row, sends the invite, publishes `vendor.status_changed`	`commission_bps` (default 1500), `agreement_ref`
`POST /api/v1/admin/vendors/{id}/reject`; `.../suspend`; `.../reactivate`	Status changes (suspend hides the vendor's items in core through the event)	`reason`
`PATCH /api/v1/admin/vendors/{id}/commission`	Change a vendor's commission for future orders; publishes `vendor.commission_changed`	`commission_bps`
`GET /api/v1/admin/products?status=pending_review`	Moderation queue	filters
`POST /api/v1/admin/products/{id}/approve`	`pending_review` to `approved`; writes `product.published` to the outbox (core then embeds it)	none

Method and path	Purpose	Input
`POST /api/v1/admin/products/{id}/reject`	To `rejected`	`reason_code`, `reason`
`GET /api/v1/admin/orders?status=`	All orders (read copy)	filters
`POST /api/v1/admin/orders/{id}/deliver`	**Calls core:** mark delivered	`note`
`POST /api/v1/admin/orders/{id}/cancel`	**Calls core:** admin cancel	`reason`
`POST /api/v1/admin/orders/{id}/payment-collected`	**Calls core:** mark cash or UPI on delivery collected	`amount_paise`, `reference`
`POST /api/v1/admin/orders/{id}/delivery`	**Calls core:** record courier and tracking	`courier_name`, `tracking_ref`
`POST /api/v1/admin/orders/{id}/refund`	**Calls core:** record or issue a refund (core calls Razorpay for online payments)	`amount_paise`, `reason`
`GET /api/v1/admin/outbox?status=failed`; `POST .../outbox/{id}/retry`	See and retry undelivered events	filters
`POST /api/v1/admin/resync`	Re-emit `product.*`, `vendor.*` events for given ids or everything	`kind`, `ids[]`
`GET /api/v1/admin/audit-log?entity_type=&entity_id=`	Audit trail	filters

7. Webhooks

Path	Service	From	Events	Verification	Status
`POST /v1/webhooks/razorpay`	**core**	Razorpay	`payment.captured`, `payment.failed`, `refund.processed`	Signature header checked; event ids stored in `idempotency_keys` so repeats do nothing	`[MVP]` built; ignored unless `online_payments_enabled`
`POST /api/v1/webhooks/whatsapp`	vendor-admin	WhatsApp provider	Delivery receipts; inbound replies such as "CONFIRM 123" `[Assumption]` (a reply becomes a command to core)	Provider signature or shared secret	`[Target]`
`POST /api/v1/webhooks/courier`	vendor-admin	Courier partner	Pickup, in transit, delivered, failed (each becomes a command to core)	Shared secret	`[Target]`

Handlers answer quickly (`200`) and do the work in a job. **Only the verified Razorpay webhook, handled in core, marks an online order paid**; Razorpay is never called from the vendor-admin service.

8. Internal API and events (between the services) `[MVP]`

Not public. Defined as OpenAPI and JSON Schema files in `contracts/` (owned by the portal repo, pinned copy in core). Background in `architecture.md` section 5.

8.1 Authentication

Every internal request and event delivery carries:

Header	Value
`X-Service`	`core` or `vendor` (the sender)
`X-Timestamp`	Unix seconds
`X-Signature`	Hex `HMAC_SHA256(secret_for_that_direction, timestamp + "." + raw body)`
`X-Request-Id`	The original request id, for tracing across both services
`X-Actor` (commands only)	`vendor_user:<id>` or `admin_user:<id>`, recorded in core's status events and audit log

The receiver rejects a timestamp more than 60 seconds old or a bad signature with `401 unauthenticated`. There is one secret per direction. Internal routes never accept user tokens and are reachable only on the Docker network (`architecture.md` section 10).

8.2 Commands: vendor-admin to core (synchronous)

Method and path	Purpose	Request	Response and errors
`PUT /internal/v1/inventory/{variant_id}`	Set `on_hand` for one variant	`on_hand`, `expected_version`	`200 {variant_id, on_hand, reserved, available, version}`. Errors: `stock_below_reserved`, `version_conflict`, `not_found`
`POST /internal/v1/inventory/bulk`	Set many	`[{variant_id, on_hand, expected_version}]`	`200` per-row results
`POST /internal/v1/orders/{id}/confirm`	`placed` to `confirmed`	`vendor_id`	`200` order state. Errors: `not_found` (also wrong vendor), `invalid_transition`, `payment_pending`, `vendor_not_approved`
`POST /internal/v1/orders/{id}/ship`	`confirmed` to `shipped`	`vendor_id`, `courier_name`, `tracking_ref`	`200`; commits the reservation
`POST /internal/v1/orders/{id}/cancel`	Vendor or admin cancel	`reason`, optional `vendor_id`	`200`; releases stock; starts a refund when money was taken
`POST /internal/v1/orders/{id}/deliver`	Admin: delivered	`note`	`200`
`POST /internal/v1/orders/{id}/payment-collected`	Admin: cash or UPI collected	`amount_paise`, `reference`	`200`; `payment_status = paid`
`POST /internal/v1/orders/{id}/delivery`	Admin: courier details	`courier_name`, `tracking_ref`	`200`
`POST /internal/v1/orders/{id}/refund`	Admin refund	`amount_paise`, `reason`	`202`; Razorpay called for online payments

Method and path	Purpose	Request	Response and errors
`POST /internal/v1/returns/{id}/decision`	Vendor decision on a return	`decision` (`approve`, `decline`), `reason`	`200`
`GET /internal/v1/versions?kind=order,inventory&cursor=`	Current versions for reconciliation	filters	`{items [{kind, id, version}], next_cursor}`
`POST /internal/v1/resend`	Ask core to re-emit order events	`kind`, `ids[]`	`202`

Every command carries an `Idempotency-Key` where repeating would matter (refund, stock bulk). Repeating a state change on an order that is already in that state returns the order with `200`. Timeouts: 3 seconds; on timeout vendor-admin returns `service_unavailable` to the portal and changes nothing.

8.3 Event delivery

Both services expose `POST /internal/v1/events` (same signing). Body is the envelope `{event_id, type, version, occurred_at, producer, payload}`. Response `200 {status}` where status is `accepted`, `duplicate` or `stale`. Any `5xx` or timeout makes the sender retry from its outbox with backoff (1 minute, 5 minutes, 30 minutes, 2 hours, 12 hours), then mark the row `failed` and alert.

8.4 Event catalog

Event	From to	Payload fields	Receiver action
`product.published`, `product.updated`	vendor-admin to core	`product_id`, `vendor_id`, `vendor_name`, `commission_bps`, `title`, `description`, `category_slug`, `category_name`, `gender`, `fabric`, `attributes`, `min_price_paise`, `max_price_paise`, `images [{url, angle, position}]`, `variants [{variant_id, size, color, price_paise, mrp_paise, is_active}]`, `returnable`, `return_window_days`, `version`	Upsert `catalog_items` (`status = approved`); create `inventory` rows for new variants (zero stock until the vendor sets it); schedule the embedding; `feed_eligible` becomes true when the embedding exists and a variant has available stock
`product.unpublished`	vendor-admin to core	`product_id`, `reason` (`archived`, `back_in_review`, `rejected_after_approval`, `vendor_suspended`), `version`	`status = hidden`, `feed_eligible = false`
`vendor.status_changed`	vendor-admin to core	`vendor_id`, `status`, `version`	Hide or show that vendor's items; block checkout for a suspended vendor
`vendor.commission_changed`	vendor-admin to core	`vendor_id`, `commission_bps`, `effective_from`, `version`	Update the rate used for new orders
`order.placed`	core to vendor-admin	Full order snapshot: ids, vendor, customer id, name, phone, address, items, prices, totals, commission, payment method and status, timers, `version`	Insert `vendor_orders` and items; email the vendor
`order.status_changed`	core to vendor-admin	`order_id`, `number`, `vendor_id`, `from_status`, `to_status`, `payment_status`, `reason`, `occurred_at`, `version`	Update the copy; notify the vendor if needed
`payment.captured`, `payment.refunded`	core to vendor-admin	`order_ids`, `amount_paise`, `reference`, `occurred_at`	Mark payment state on the copy; feeds statements
`order.confirmation_overdue`	core to vendor-admin	`order_id`, `vendor_id`, `due_at`	Send the reminder email
`customer.deleted`	core to vendor-admin	`customer_id`	Blank name, phone and address on that customer's orders

Rules: events are add-only (adding a field is allowed; removing one needs a new `type` version); the receiver ignores an event whose `version` is not higher than the stored one; ordering between different entities is not guaranteed and must not be assumed. The customer push notification is sent by **core** at the moment it changes an order status; it does not depend on any event.

9. Rate limits and idempotency

Operation	Service	Limit (per)	Idempotency
`guestStart`	core	5 per hour per IP	Device id reuses the existing guest
Customer `login`	core	10 per minute per IP and 5 per hour per account	n/a
`signUp`	core	10 per hour per IP	n/a
Vendor `login`	vendor-admin	10 per minute per IP and 5 per hour per account	n/a
Admin `login`	vendor-admin	5 per minute per IP	n/a
`POST /public/vendor-applications`	vendor-admin	5 per hour per IP; one open application per email	Same email returns the existing application
`POST /public/contact`	core	5 per hour per IP	n/a
`swipeRecord`	core	300 per minute per user	`clientEventId`
`orderPlace`	core	10 per hour per user	`Idempotency-Key` (required), unique per customer
Razorpay webhook	core	Provider IPs only plus signature	Event id
Image presign	vendor-admin	60 per hour per vendor	n/a
Stock set	vendor-admin	120 per minute per vendor	`expected_version`
Internal API	both	600 per minute per direction	Command key, `event_id`

Operation	Service	Limit (per)	Idempotency
Others	each	120 per minute per user	Optional `Idempotency-Key`

These numbers are starting values `[Assumption]`; Cloudflare rules add a first layer.

10. MVP slice `[MVP]`

The MVP keeps the **app on REST** at today's `/v1` routes on **core** and adds the commerce, public and webhook routes at paths that match this design (only the prefix changes later). Checkout, orders and stock are core routes with no call to the vendor-admin service. The vendor portal, admin and vendor application are on the **vendor-admin** service from the start; its order and stock routes call core. The full list, exact field names and schedule are in `06-mvp-slice-and-migration.md`. App operations map to today's routes as follows:

App operation (target)	Today's REST route `[As-is]` on core
`guestStart`	`POST /v1/auth/shadow`
`signUp`	`POST /v1/auth/signup`
`login`, `googleLogin`	`POST /v1/auth/login`, `POST /v1/auth/google-login`
`accountDelete`	`POST /v1/auth/delete` (also emits `customer.deleted`)
`feed`	`POST /v1/items/getinitial`
`product` and lists	`GET /v1/items/getall`
`swipeRecord`	`POST /v1/user/swipes`
`closets`, `closetCreate`, `closetAddItem`, `closetJoin`, `closetLeave`, `closetRemoveItem`, `closetAddToCart`	`GET /v1/closets/<user_id>/`, `POST /v1/closets/create/`, `add-item/`, `add-collab`, `delete`, `delete-item`, `add-to-cart`
`cart`, `cartAdd`, `cartRemove`	`GET /v1/cart/<user_id>/`, `POST /v1/cart/add/`, `POST /v1/cart/remove/`
`addresses`, `addressSave`	`GET /v1/address/<user_id>/`, `POST /v1/address/`
`deviceRegister`	`POST /v1/user/register_fcm`
`orders`, `order`	`GET /v1/order/user/<user_id>` becomes `GET /v1/orders/`, `GET /v1/orders/{id}/`
`orderPlace`	`POST /v1/order-db/` is retired; new `POST /v1/orders/` (a single core transaction)
`orderCancel`	new `POST /v1/orders/{id}/cancel/`
`publicConfig`, `serviceability`	new `GET /v1/public/config`, `GET /v1/serviceability`
Razorpay webhook	new `POST /v1/webhooks/razorpay` (inactive while the switch is off)

In the MVP these routes also stop trusting `user_id` from the request body (the token identifies the user), which changes the request bodies; the app and core are released together. Vendor and admin routes have no legacy: they are new in the vendor-admin service.

11. Security checklist for the APIs

- Tokens on every route except the public list, webhooks (signed), internal routes (HMAC) and health.
- Separate signing keys for core and vendor-admin; audience checked on every request; a customer token cannot call the vendor-admin service and a vendor token cannot call core.
- Razorpay secret and webhook secret exist **only in core's** settings; the vendor-admin service has no payment credentials.
- CORS allow-list: on vendor-admin only the portal and admin origins; on core only the website's swipe-demo origin; mobile apps do not use CORS.
- No CSRF tokens needed for bearer-token APIs; SQLAdmin keeps its own session and CSRF protection and sits behind Cloudflare Access.
- Presigned or signed upload URLs expire in minutes and are locked to the declared type and size.
- Webhook bodies are verified before parsing; events are stored once.
- Internal HMAC with a 60 second window; `/internal/*` blocked on every public hostname and reachable only on the Docker network; commands record `X-Actor` in core's audit log.
- GraphQL (core): persisted queries, depth and alias limits, no introspection in production.
- Logs never contain passwords, tokens, HMAC secrets, card or payment details, or message bodies; `X-Request-Id` is the shared trace key.

12. Testing and clients

- Typed clients are generated: GraphQL types for the app (schema export plus codegen), an OpenAPI client for the portal (from the vendor-admin OpenAPI file).
- **Contract tests** run in CI in both repos against `schema.graphql`, the vendor-admin `openapi.json`, `internal-api.openapi.json` and the event JSON Schemas. Vendor-admin tests call a stub core built from the contract; core tests post sample events to a stub vendor-admin receiver.
- **Concurrency tests** in core: many parallel checkouts for the last unit; webhook replay; the same `Idempotency-Key` twice.
- An **end-to-end script** on staging places an order, confirms and ships it through the vendor-admin API, marks it delivered and collected, and checks both services' copies and the customer push; it runs before each release.
- Each endpoint has tests for success, wrong role or wrong audience, someone else's object (expects 404), bad input, and (for writes) a repeat with the same idempotency key.

13. Open items

Final REST versus GraphQL for the app (core only); the portal fallback; WhatsApp inbound replies as a confirm channel; customer return endpoints on the MVP; share links by invite token versus closet id during the transition; whether the unpaid online order window is 15 minutes; stock reservation expiry for cash orders (Q40);

admin order screens UI (Q38). Tracked in `open-questions.md`. Decided and closed: commerce and stock in core, vendor identity separate, portal in React, product edit review rules, commission (15%), return window (7 days), confirmation window (4 business hours).